Storing user generated content

Last reviewed: 6 months ago

Introduction

User generated content (UGC) is an essential aspect of modern applications. This includes users uploading profile photos, documents, and videos, as well as AI models generating images, summaries, or structured data. Therefore, applications require a reliable, scalable, and cost-effective solution for storing and accessing this content.

Cloudflare R2 is an S3-compatible object storage with zero egress fees, making it ideal for handling content uploads and delivery at scale. Combined with Cloudflare Workers and Cloudflare's global network, it enables fast, secure, and cost-effective workflows for ingesting and managing UGC.

This reference architecture explores two common UGC workflows, both optimized for performance, security, and cost efficiency:

1. **Secure User Uploads to R2 via Signed URLs:** Allowing users to upload files (profile images, documents, etc.) efficiently and securely without overloading backend systems.
2. **AI-Generated Content Stored in R2:** Storing content generated by Workers AI or external AI services, ensuring inference results are persistently available for future use.

Use Cases

Use Case 1: Secure User Uploads to R2 via Signed URLs

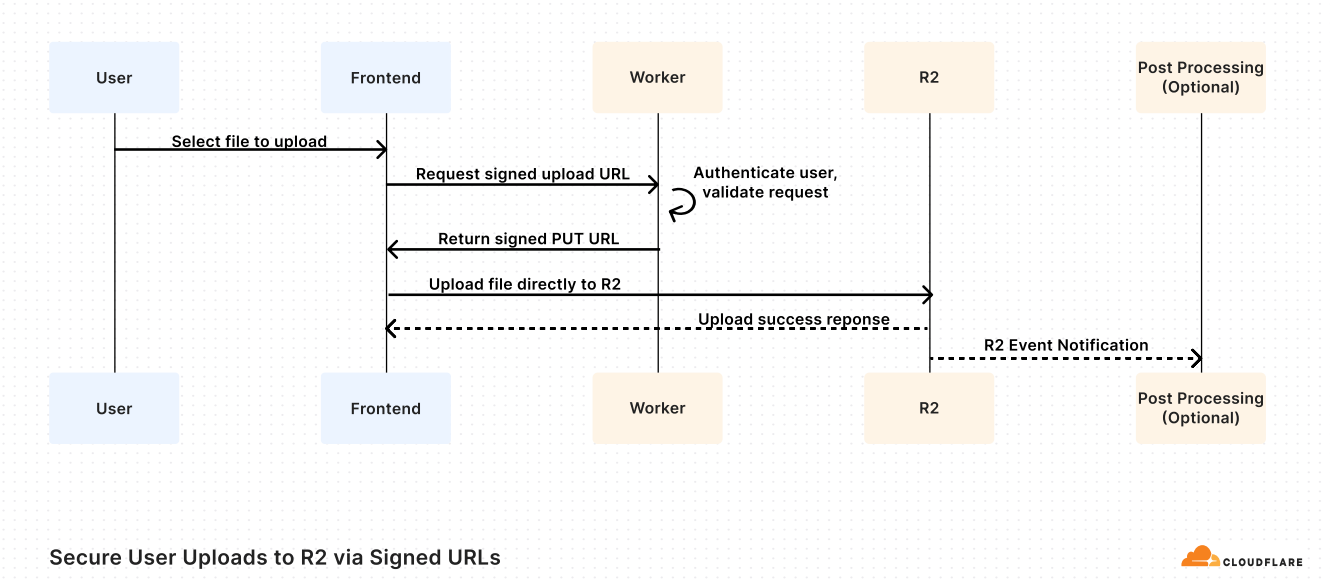
User generated content typically starts with file uploads, including profile pictures, resumes, rich media, and documents. Applications must securely validate and store these uploads while avoiding latency, high costs, and unnecessary complexity in the backend.

In this architecture, we use **R2** as the primary storage layer and a **Worker** to control upload access. Files are uploaded directly from the user's browser or device to R2 using signed URLs, which are generated by the Worker after validating the user's permissions and upload intent.

This approach avoids routing large files through the application backend or Worker, reducing latency and operational cost—while ensuring tight control over access and security.

And because R2 is natively integrated with Cloudflare's global network, files stored in R2 are accessible with

low latency from anywhere in the world—and **without any egress fees**, even as your application scales.



Use Case 1: Secure User Uploads to R2 via Signed URLs

How it Works

1. **User initiates upload from the frontend:** The app collects file details (e.g. size, name) and calls a backend API (a Cloudflare Worker) to begin the upload process.
2. **Worker authenticates the user and validates the request:** The Worker confirms that the user is logged in, has upload permissions, and that the file is within acceptable limits (for example, 10MB max, allowed MIME types).

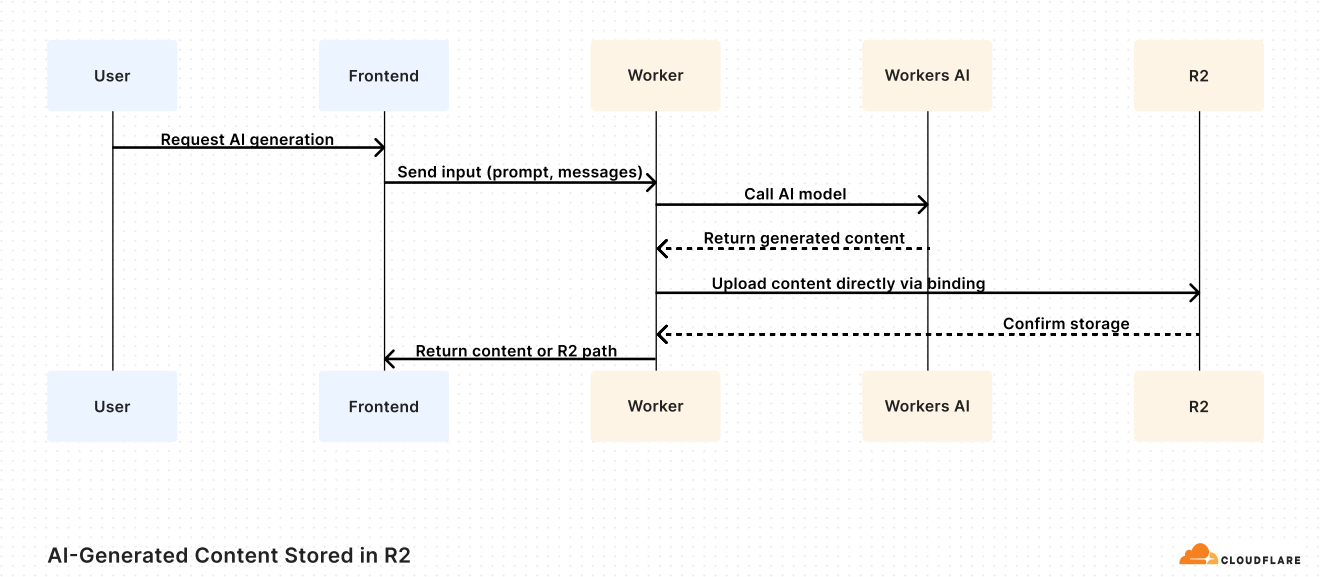
3. **Worker returns a signed PUT URL to R2:** A signed URL allows the frontend to upload directly to R2 for a limited time, under a specific key or namespace. There is no need for the Worker to handle large files directly.
4. **Frontend uploads the file directly to R2:** The file is streamed directly from the client to R2.
5. **(Optional) Trigger post-upload workflows:** R2 offers event notifications to send messages to a queue when data in your R2 bucket changes, like a new upload. Example post-processing:
    - Scan, moderate, or transform the file.
    - Write metadata (for example, `user_id`, `file_path`, `timestamp`) to D1, Cloudflare's serverless SQL database.
    - Notify the user or update a dashboard/UI.

For more information on uploading data directly from the client to R2, refer to the documentation on presigned URLs.

Use Case 2: AI-Generated Content Stored in R2

Many modern applications utilize AI-generated content, which can include product descriptions, profile pictures, audio clips, and more. When this content is created in response to user actions or scheduled events, it must be stored immediately, reliably, and at scale.

This architecture employs Workers AI to perform inference at the edge and then stores the generated output directly in Cloudflare R2, all within a single Worker.



Use Case 2: AI-Generated Content Stored in R2

How it Works

1. **User initiates content generation:** The frontend sends a request to a Cloudflare Worker to create content using an AI model (for example, "Create a thumbnail image for this product").
2. **Worker invokes Workers AI:** The Worker passes the user input to a model deployed on Workers AI.
3. **Generated output is returned to the Worker:** The response could be plain text, a Base64 image, a binary buffer, or other structured data—depending on the model type.
4. **Worker uploads the output to R2 directly:** No signed URL or client upload is needed. The Worker performs a secure, authenticated PUT request to store the output in a designated bucket.
5. **Worker returns success and metadata to the frontend:** The client receives a reference to the

stored file (such as a path, object key, or signed download URL if needed).

Refer to Use R2 from Workers for more information on accessing R2 buckets via Cloudflare Workers.

Summary

By storing **user-generated content in Cloudflare R2**, applications gain:

- A highly scalable storage backend
- Fast access through Cloudflare's edge computing
- Predictable costs with zero egress fees
- Seamless AI + UGC workflows that maximize efficiency

This architecture ensures that content is stored, processed, and delivered **fast, securely, and cost-effectively**.

Related Links

- Cloudflare R2 Product Page
- R2 Presigned URLs
- Use R2 from Workers
- Migrating Data to R2
- Event notifications for storage reference architecture
- Why choose Cloudflare R2 vs Amazon S3 ↗

Was this helpful?

Previous
← On-demand Object Storage Data Migration

Next
Cloudflare's benefits for SaaS providers →

Last updated: Aug 19, 2025

Resources **Support**